Report di Cybersecurity

ANALISI SAST

Target: OWASP Juice Shop Repository

Autore: Stefano Panosetti

Ruolo: Studente di ICT

Istituto: ITS Academy

Data di Consegna: 3 giugno 2026

Versione Doc: 2.0

TABELLA STORICO REVISIONI

Versione	Data	Autore	Tipo Modifica	Descrizione della Modifica
1.0	27 maggio 2026	Stefano Panosetti	Creazione	Stesura iniziale dell'introduzione e raccolta casi pratici dal software Juice Shop.
1.1	30 maggio 2026	Stefano Panosetti	Modifica	Revisione dei contenuti e aggiunta di due casi studio
1.2	01 giugno 2026	Stefano Panosetti	Approvazione	Considerazioni Finali e rifiniture Layout
2.0	03 giugno 2026	Stefano Panosetti	Pubblicazione	Report Pubblicato

INDICE GENERALE DEL REPORT

1. EXECUTIVE SUMMARY 3

1.1 Analisi Generale e Obiettivi di Sicurezza 3

1.2 Quadro delle Vulnerabilità Rilevate 3

2. METODOLOGIA E MODELLO DI RIFERIMENTO 4

2.1 Principi di Sicurezza (La Triade CIA) 4

2.2 Fasi dell'Attività Organizzata 5

2.3 Glossario e Legenda Universale dei Termini Tecnici 6

3. GLI STRUMENTI UTILIZZATI: ARCHITETTURA SONARQUBE 7

3.1 Il Ruolo di SonarQube e SonarScanner 7

3.2 Flusso Operativo della Scansione e Gestione dei Token 7

4. ANALISI TECNICA ED ESEMPI DI REMEDIATION (Technical Report) 8

4.1 Caso Tecnico 1: Esposizione di Chiave Privata Crittografica (CWE-798 / OWASP A02) 8

4.2 Caso Tecnico 2: Credenziali di Testing in chiaro nel Front-End (CWE-798 / OWASP A07) 9

4.3 Caso Tecnico 3: XSS (Cross-Site Scripting) CWE 79 10

4.4 Caso Tecnico 4: Remote Code Execution (RCE) CWE-95 11

4.5 Caso Tecnico 5: Denial of Service (DoS) 12

5. CONSIDERAZIONI FINALI 13

1. EXECUTIVE SUMMARY

1.1 Analisi Generale e Obiettivi di Sicurezza

La cybersecurity è il ramo dell'informatica che si concentra sullo studiare, testare e risolvere quelli che sono i problemi relativi alla sicurezza di un software, come per esempio l'esposizione dei dati sensibili o problemi relativi alla gestione dei flussi economici. Anche durante lo sviluppo di un software, specialmente nel 2026, bisogna testare continuamente i vari problemi relativi alla sicurezza poiché sono il pilastro fondamentale su cui si basa tutta la nostra infrastruttura. Oggi, infatti, strumenti di analisi del codice vengono integrati direttamente nel **flusso di sviluppo** (pipeline CI/CD), ci assicurano che il codice che abbiamo in produzione sia sicuro e che possiamo continuare a svilupparlo senza accumulare problemi strutturali.
Il presente documento raccoglie gli esiti dell'attività di security assessment condotta sul repository del progetto **Juice Shop**, una delle applicazioni web più note e intenzionalmente vulnerabili utilizzate come benchmark a livello globale nel settore della sicurezza informatica. L'applicazione è scritta in TypeScript ed utilizza Angular (uno dei framework più usati nello sviluppo frontend)

1.2 Quadro delle Vulnerabilità Rilevate

La scansione globale del codice sorgente ha evidenziato una superficie d'attacco particolarmente estesa, con metriche critiche che richiedono interventi immediati di rimediazione (Remediation) per evitare la compromissione totale dell'applicazione.

Metrica di Sicurezza / Categoria	Rilevazioni / Valore	Rating / Grado di Rischio Complessivo
Security Open Issues (Vulnerabilità attive)	*14 Issues*	**Rating E (Massima Gravità)**
Security Hotspots (Codice da verificare nel contesto)	*107 Hotspots*	**Rating E (Revisione Richiesta)**
Reliability Open Issues (Affidabilità / Bug logici)	*153 Issues*	**Rating D (Affidabilità Scarsa)**
Maintainability Open Issues (Code Smell / Pulizia)	*4.9k Issues*	**Rating A (Ottimo stato generale)**
Coverage (Copertura dei Test automatici)	*0.0% su 33k righe*	**Grave mancanza di Unit Test**
Duplications (Porzioni di codice duplicate)	*9.5% su 117k righe*	**Sotto la soglia limite (10%)**

2. METODOLOGIA E MODELLO DI RIFERIMENTO

2.1 Principi di Sicurezza - La Triade CIA

Qualsiasi analisi informatica presuppone la tutela di tre capisaldi, la cui violazione determina una vulnerabilità del sistema:

- **Riservatezza (Confidentiality):** l'accesso ai dati sensibili è permesso soltanto agli utenti autorizzati all'interno del sistema.
- **Integrità (Integrity):** anche la modifica o l'alterazione dei file deve essere gestita soltanto dagli amministratori, quindi previo consenso.
- **Disponibilità (Availability):** Il software deve essere accessibile ogni volta che l'utente autorizzato ne ha la necessità.

2.2 Fasi dell'Attività Organizzata

L'attività di assessment si è strutturata in quattro fasi metodologiche disposte in ottica consulenziale:

1. **Pianificazione (Planning):** Definizione del perimetro dell'analisi *(Repository di OWASP Juice Shop in ambiente isolato)* e configurazione dell'infrastruttura locale SonarQube.
2. **Esecuzione (Execution):** Lancio del motore SonarScanner sulla directory sorgente per l'estrazione automatica dei pattern non conformi e delle violazioni di sicurezza.
3. **Revisione (Review):** Analisi manuale delle Criticità e dei Security Hotspots per escludere i falsi positivi, isolando le issue bloccanti e studiando i meccanismi di attacco reali.
4. **Reporting:** Redazione del presente documento definitivo volto a illustrare e a documentare le corrette strategie di Remediation

2.3 Glossario e Legenda Universale dei Termini Tecnici

Di seguito viene strutturata la tabella contenente la legenda e la spiegazione di tutti i principali termini tecnici e acronimi utilizzati nell'ambito della sicurezza informatica applicata in questa relazione:

Termine / Acronimo	Definizione e Significato Operativo nel Contesto Cyber
VULNERABILITY	In informatica una *vulnerabilità* è un difetto, una falla o un errore nel codice sorgente o nell'architettura di un software che può essere sfruttato dagli hacker (o dai penetration tester) per compiere operazioni illecite e critiche nel sistema stesso.
EXPLOIT	Un *exploit* è un segmento di codice o un programma che sfrutta (esegue con successo) malevolmente vulnerabilità o difetti di sicurezza nel software o nell'hardware per infiltrarsi e avviare un attacco. L'exploit non è il malware stesso, ma viene utilizzato per distribuirlo.
REPOSITORY	Un *repository* (spesso abbreviato in "repo") contiene l'intero codice sorgente di un'applicazione e l'intera cronologia di tutte le modifiche apportate.
GIT	*Git* è software di versioning, cioè funziona come una macchina del tempo dei progetti. Ci permette di effettuare salvataggi (snapshot), detti commit, grazie è possibile confrontare le modifiche nuove con versioni precedenti e/o ripristinarle
FRONTEND	Tutta parte visibile di un sito web, di un software o di un'applicazione con cui interagisci direttamente. Include l'interfaccia utente ed è gestita principalmente da HTML, CSS e JAVASCRPIT (i linguaggi del web)
BACKEND	È la parte "invisibile" di un software, di un'app o di un sito web. Gestisce tutta la business logic e le comunicazioni con i database. Node.js è un esempio di backend che verrà preso in analisi successivamente
SAST	*Static Application Security Testing*. Tecnica di analisi che consiste nell'analizzare il codice sorgente senza compilarlo, mappando la sintassi e come si muovono i dati nel sistema, cercando di tracciare possibili bug strutturali. La SAST può essere svolta sia manualmente che strumentalmente, con apposti software come SonarQube.
DAST	*Dynamic Application Security Testing*. Analisi dinamica eseguita con l'applicazione in stato di esecuzione. Simula gli attacchi di un hacker esterno (es. invio di input malevoli tramite proxy come BurpSuite) valutando la risposta del server.
CVE	*Common Vulnerabilities and Exposures*. Registro standard internazionale pubblico che assegna un codice identificativo univoco ad una specifica falla di sicurezza reale scoperta in un software specifico. Una CVE famosissima è per esempio CVE-2021-44228 (falla Log4Shell) con un punteggio di CVSS di 10.0.

CVSS	*Common Vulnerability Scoring System.* Sistema di calcolo standardizzato usato per assegnare un punteggio numerico da 0.0 a 10.0 alla gravità di una CVE, basandosi su metriche come la facilità di attacco e l'impatto sul sistema.
CWE	*Common Weakness Enumeration*. Dizionario internazionale gestito dalla MITRE Corporation che cataloga i difetti logici, strutturali e di progettazione del codice. Descrive il pattern di errori commesso dallo sviluppatore (es. le password scritte nel codice).
OWASP	*Open Web Application Security Project*. Organizzazione internazionale no-profit dedicata a migliorare la sicurezza del software applicativo web. Rilascia gratuitamente standard, linee guida e framework metodologici di testing.
OWASP Top 10	Classifica periodica rilasciata da OWASP che elenca i 10 rischi più diffusi e impattanti riscontrati sulle applicazioni web a livello mondiale.
CHIAVE RSA	Una chiave RSA è uno strumento matematico utilizzato nella crittografia per proteggere i dati trasmessi su reti come Internet. Serve a garantire la riservatezza delle informazioni (Confidentiality) e ad assicurare che un messaggio venga da un mittente specifico (firma digitale)
SECURITY OPEN ISSUE	Vulnerabilità certa e sfruttabile, richiede revisione e solitamente non si tratta mai di falsi positivi.
SECURITY HOTSPOTS	E' un punto critico del codice, ma occorre essere revisionato da un umano per poter decidere se è davvero rischioso o no, a seconda del contesto in cui si trova
SECRET VAULTING	Processo di archiviazione, gestione e protezione centralizzata delle credenziali digitali sensibili, come chiavi API, password, token e certificati. Sostituisce la pratica non sicura di inserire le password direttamente nel codice sorgente
SCRIPT	Insieme di istruzioni scritte in un linguaggio di scripting (JavaScript, Python, ecc..) che può essere utilizzato in moltissimi campi. Lo scopo principale è creare automazioni
XSS	*Cross-Site Scripting*. È una vulnerabilità di sicurezza informatica che permette a malintenzionati di inserire ed eseguire script dannosi (generalmente JavaScript) all'interno di siti o applicazioni web
RCE	*Remote Code Execution*, Permette a un utente malintenzionato di eseguire codice direttamente su un computer o su un server al quale non ha accesso fisico (remoto)
SANDBOX	In informatica è un Ambiente di test o ambiente virtuale isolato, quindi un'area protetta per testare codice o isolare minacce
DOS	*Denial of Service*, è un attacco che mira a sovraccaricare un Web Server con traffico per interromperne le normali operazioni.

3. GLI STRUMENTI UTILIZZATI: ARCHITETTURA SONARQUBE

3.1 Il Ruolo di SonarQube e SonarScanner

In questo assessment, l'analisi automatizzata delle debolezze è stata affidata alla suite di **SonarQube**, uno degli strumenti più utilizzati per fare Analisi SAST. Il software funziona analizzando la codebase attraverso regole preconfigurate appartenenti a standard industriali, permettendo di intercettare non solo i bug strutturali, ma anche le violazioni di sicurezza che espongono l'applicazione a potenziali exploit.

L'architettura si divide fondamentalmente in due componenti che lavorano in sinergia:

- **SonarQube Server:** Il motore centrale che risiede localmente (nel nostro laboratorio configurato ed esposto sulla porta di loopback *http://localhost:9000*). Gestisce il database delle regole, elabora i report inviati dagli scanner e fornisce la dashboard grafica per l'ispezione analitica delle problematiche rilevate
- **SonarScanner:** Lo strumento a riga di comando (CLI) che viene lanciato direttamente all'interno della cartella principale del codice sorgente del progetto (in questo caso, la repository di Juice Shop). Effettua la scansione fisica della codebase e spedisce l'estratto al Server centrale di SonarQube per l'elaborazione e la revisione dei problemi mappati

3.2 Flusso Operativo della Scansione e Gestione dei Token

Il flusso operativo implementato sul mio terminale ha seguito i seguenti passaggi fondamentali scritti a mano, documentando la corretta prassi di autenticazione ed esecuzione della pipeline:

Per avviare la comunicazione sicura tra l'ambiente locale in cui risiede il codice e la dashboard di monitoraggio, è stato necessario accedere al pannello di amministrazione di SonarQube su localhost per generare una chiave d'accesso crittografica univoca, denominata **Token di Autenticazione**. Questo approccio evita l'inserimento di credenziali amministrative in chiaro negli script di scansione, proteggendo l'accesso al server.

Una volta configurato il file di proprietà *sonar-project.properties* all'interno del progetto **con le specifiche del target (token del progetto, nome identificativo e percorsi dei sorgenti)**, lo scanner è stato invocato tramite terminale passando il token generato, il nome del progetto e l'indirizzo del server con la porta 9000. Al termine dell'elaborazione, i risultati sono stati aggregati automaticamente sulla dashboard

4. ANALISI TECNICA ED ESEMPI DI REMEDIATION (Technical Report)

4.1 Caso Tecnico 1: Esposizione di Chiave Privata

- **File Coinvolto:** *juice-shop-master/lib/insecurity.ts*
- **Identificazione CWE:** **CWE-798 - Use of Hard-coded Credentials**
- **Mappatura OWASP: A02:2021 – Cryptographic Failures**
- **Severità:** Bloccante (Critical)
- **Tipo: Security Open Issue**

Descrizione della Vulnerabilità:

All'interno del file di gestione della sicurezza del back-end, è stato commesso l'errore critico di definire esplicitamente la variabile ***const privateKey*** inserendo l'intera chiave **privata RSA** sotto forma di stringa testuale fissa (cablata a riga 23 del codice originale). Questa pratica espone il sistema a un rischio altissimo: chiunque abbia accesso in lettura al repository su Git entra in possesso della chiave segreta dell'applicazione. Con questa chiave qualunque malintenzionato è in grado di decifrare il traffico protetto o, peggio, di falsificare e firmare autonomamente i token di autenticazione, violando l'accesso con massimi privilegi amministrativi ed eludendo qualunque controllo.

```
21
22  export const publicKey = fs ? fs.readFileSync('encryptionkeys/jwt.pub', 'utf8') : 'placeholder-public-key'
23  const privateKey = '-----BEGIN RSA PRIVATE KEY-----\r\nMIICXAIBAAKBgQDNwqLEe9wgTXCbC7
    +RPdDbBbeqjdbs4kOPOIGzqLpXvJXlxxW8iMz0EaM4BKUqYsIa+ndv3NAn2RxCd5ubVdJJcX43zO6Ko0TFEZx/65gY3BE0O6syCEmUP4qbSd6exou/F
    +WTISzbQ5FBVPVmhnYhG/kpwt/cIxK5iUn5hm+4tQIDAQABAoGBAI+8xiPoOrA+KMnG/T4jJsG6TsHQcDHvJi7o1IKC/
    hnIXha0atTX5AUkRRce95qSfvKFweXdJXSQ0JMGJyfuXgU6dI0TcseFRfewXAa/ssxAC
    +iUVR6KUMh1PE2wXLitfeI6JLvVtrBYswm2I7CtY0q8n5AGimHWVXJPLfGV7m0BAkEA+fqFt2LXbLtyg6wZyxMA/
    cnmt5Nt3U2dAu77MzFJvibANUNHE4HPLZxjGNXN
    +a6m0K6TD4kDdh5HfUYLWWRBYQJBANK3carmulBwqzcDBjsJ0YrIONBpCAsXxk8idXb8jL9aNIg15Wumm2enqqObahDHB5jnGOLmbasizvSVqypfM9UCQCQl
    8xIqy+YgURXzXCN+kwUgHinrutZms87Jyi+D8Br8NY0+Nlf+zHvXAomD2W5CsEK7C+8SLBr3k/
    TsnRWHJuECQHFE9RA2OP8WoaLPuGCyFXaxzICThSRZYluVnWkZtxsBhW2W8z1b8PvWUE7kMy7TnkzeJS2LSnaNHoyxi7IaPQUCQCWWU4U+v4lD7uYBw00Ga/
    xt+7+UqFPlPVdz1yyr4q24Zxaw0LgmuEvgU5dycq8N7JxjTubX0MIRR+G9fmDBBl8=\r\n-----END RSA PRIVATE KEY-----'
24
```

Soluzione Applicata ed Implementazione della Remediation:

In cybersecurity, un dato segreto compromesso all'interno della Git History non può essere messo in sicurezza semplicemente nascondendolo o rimuovendolo, poiché il controllo versione mantiene traccia dei commit passati. La rimediazione ha richiesto la revoca immediata del segreto e la sua esternalizzazione nel codice. Sfruttando l'architettura *Node.js* e il modulo nativo *fs(File System)*, la chiave privata è stata rimossa fisicamente dal sorgente, salvata in una cartella protetta del server e **richiamata dinamicamente a runtime tramite la funzione sicura *readFileSync***, simulando lo stesso modello protetto già implementato dagli sviluppatori per la chiave pubblica.

```
21
22  export const publicKey = fs ? fs.readFileSync('encryptionkeys/jwt.pub', 'utf8') : 'placeholder- public-key';
23  // SOLUTION: La chiave privata appena ricreata viene letta in modo sicuro dal file system sul server senza essere
    esposta
24  export const privateKey = fs ? fs.readFileSync('encryptionkeys/jwt.key', 'utf8') : 'placeholder-private-key';
25
26
```

4.2 Caso Tecnico 2: Credenziali in chiaro nel Front-End

- **File Coinvolto:** *juice-shop-master/frontend/src/app/login/login.component.ts*
- **Identificazione CWE: CWE-798 - Use of Hard-coded Credentials**
- **Mappatura OWASP: A07:2021 – Identification and Authentication Failures**
- **Severità:** Bloccante (Critical)
- **Tipo: Security Open Issue**

Descrizione della Vulnerabilità:

Nel componente di login TypeScript dell'interfaccia utente, sono state inserite in chiaro una mail di test e la password in chiaro adibita alle verifiche d'ambiente. **Questo errore evidenzia l'utilità della classificazione CWE:** nonostante i file coinvolti e i contesti siano diversi (un file backend nel Caso 1 e un componente frontend nel Caso 2), la debolezza logica è esattamente la stessa. L'impatto qui è amplificato dal fatto che il codice front-end viene interamente compilato e distribuito in bundle JavaScript scaricati dal browser dell'utente finale. Qualsiasi utente, ispezionando i sorgenti tramite i Developer Tools del browser, può estrarre la password e violare i profili interni del sistema cliente.

```
60  public clientId = '1005568560502-6hm16lef8oh46hr2d98vf2ohlnj4nfhq.apps.googleusercontent.com'
61  public oauthUnavailable = true
62  public redirectUri = ''
63  public testingUsername = 'testing@juice-sh.op'
64  public testingPassword = 'IamUsedForTesting'
```

Soluzione Applicata in fase di Sviluppo:

Per ottimizzare le attività del team evitando dati ridondanti, le stringhe statiche sono state rimosse dalle variabili locali del componente e centralizzate sfruttando il file **environment.ts**

```
61  public clientId = environment.googleClientId;
62  public oauthUnavailable = true;
63  public redirectUri = '';
64  public testingUsername = environment.testUser;
65  public testingPassword = environment.testPassword;
```

```
1  export const environment = {
2    production: false,
3    googleClientId: '1005568560502-6hm16lef8oh46hr2d98vf2ohlnj4nfhq.apps.googleusercontent.com',
4    testUser: 'testing@juice-sh.op',
5    testPassword: 'IamUsedForTesting'
6  };
```

Nota fondamentale sulla Produzione:

È importantissimo evidenziare che lo spostamento delle credenziali nel file environment.ts risolve la ridondanza in fase di sviluppo locale, ma non è idoneo per l'ambiente di produzione reale. Poiché Angular compila il codice in bundle destinati al client, **l'unico modo per proteggere i dati sensibili è iniettarli direttamente dal server di back-end** tramite sistemi protetti di Secret Vaulting

4.3 Caso Tecnico 3: XSS (Cross-Site Scripting)

- **File Coinvolto:** *juice-shop-master/frontend/src/app/about/about.component.ts*
- **Identificazione CWE: CWE-79: Improper Neutralization of Input During Web Page Generation**
- **Mappatura OWASP: A03:2021 – Injection**
- **Review priority:** High
- **Tipo: Security Hotspot**

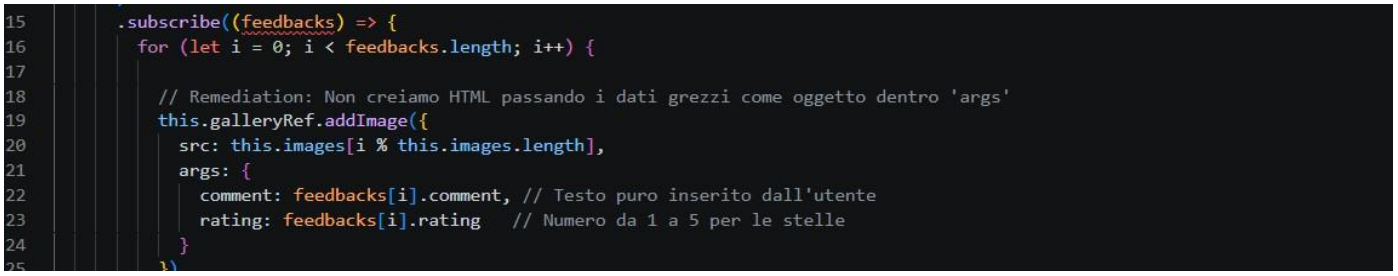
Descrizione della Vulnerabilità:

Nel componente About è stato segnalato un errore sulla compilazione dei campi: Gli sviluppatori hanno utilizzando la funzione di Angular **bypassSecurityTrustHtml** per renderizzare a schermo tutti i commenti ricevuti. Facendo questo Angular bypassa la sua funzione di "sterilizzazione" del code (neutralizzazione di tag HTML e Script Malevoli) permettendo l'inserimento di codice malevolo nel database, e rieseguendolo ogni volta che un utente visita la pagina. Questo tipo di vulnerabilità è chiamato **Stored XSS** ed è molto pericoloso perché permette di rubare token di autenticazione o compiere azioni per conto della vittima.



```
104  populateSlideshowFromFeedbacks () {
105    this.feedbackService
106      .find()
107      .pipe(
108        catchError((err) => {
109          console.error(err)
110          return EMPTY
111        })
112      )
113      .subscribe((feedbacks) => {
114        for (let i = 0; i < feedbacks.length; i++) {
115
116          feedbacks[i].comment = `<figcaption><p class="feedback-comment">${
117            feedbacks[i].comment
118          }</p><div class="feedback-stars">(${this.stars[feedbacks[i].rating]})</div></figcaption>`
119          feedbacks[i].comment = this.sanitizer.bypassSecurityTrustHtml(
120            feedbacks[i].comment
121          )
122
123          this.galleryRef.addImage({
124            src: this.images[i % this.images.length],
125            args: feedbacks[i].comment
126          })
127        }
```

Soluzione Applicata ed Implementazione della Remediation:



```
15      .subscribe((feedbacks) => {
16        for (let i = 0; i < feedbacks.length; i++) {
17
18          // Remediation: Non creiamo HTML passando i dati grezzi come oggetto dentro 'args'
19          this.galleryRef.addImage({
20            src: this.images[i % this.images.length],
21            args: {
22              comment: feedbacks[i].comment, // Testo puro inserito dall'utente
23              rating: feedbacks[i].rating   // Numero da 1 a 5 per le stelle
24            }
25          })
```

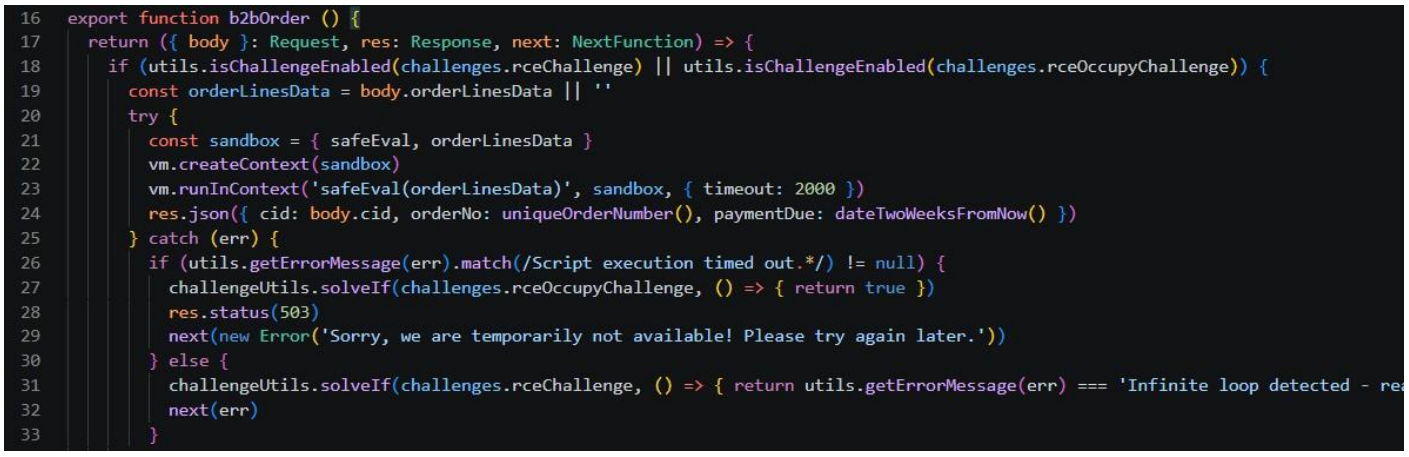
Per mettere in sicurezza il componente, **il codice è stato ristrutturato eliminando completamente l'uso del metodo bypassSecurityTrustHtml e l'uso dell'oggetto DomSanitizer** per questa operazione.
Se un utente tenta di inserire del codice maligno, questo viene renderizzato come semplice testo innocuo e non viene interpretato dal browser.

4.4 Caso Tecnico 4: Remote Code Execution (RCE)

- **File Coinvolto:** *juice-shop-master/routes/b2bOrder.ts*
- **Identificazione CWE: CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code ('Eval Injection')**
- **Mappatura OWASP: A03:2021 – Injection**
- **Review priority:** Medium
- **Tipo: Security Hotspots**

Descrizione della Vulnerabilità:

A riga 23 del componente b2bOrder (serve per ordini B2B) è stato trovato una falla di sicurezza relativa alla RCE. Viene create la costante orderLinesData prendendo come oggetto una richiesta e mandandola al server direttamente, senza eseguire controllo preventivo o coercizione, tramite comando **vm.runInContext('safeEval(orderLinesData)'**. Nonostante gli sviluppatori abbiano utilizzato notevil (una libreria che dovrebbe eseguire codice in modo sicuro) e creato una sandbox, questo non basta per essere protetti, infatti notevil stesso ha presentato vulnerabilità e le Sandbox in node.js tendono a non essere sicure e vengono bucate spesso



```
16  export function b2bOrder () {
17    return ({ body }: Request, res: Response, next: NextFunction) => {
18      if (utils.isChallengeEnabled(challenges.rceChallenge) || utils.isChallengeEnabled(challenges.rceOccupyChallenge)) {
19        const orderLinesData = body.orderLinesData || ''
20        try {
21          const sandbox = { safeEval, orderLinesData }
22          vm.createContext(sandbox)
23          vm.runInContext('safeEval(orderLinesData)', sandbox, { timeout: 2000 })
24          res.json({ cid: body.cid, orderNo: uniqueOrderNumber(), paymentDue: dateTwoWeeksFromNow() })
25        } catch (err) {
26          if (utils.getErrorMessage(err).match(/Script execution timed out.*/) != null) {
27            challengeUtils.solveIf(challenges.rceOccupyChallenge, () => { return true })
28            res.status(503)
29            next(new Error('Sorry, we are temporarily not available! Please try again later.'))
30          } else {
31            challengeUtils.solveIf(challenges.rceChallenge, () => { return utils.getErrorMessage(err) === 'Infinite loop detected - re
32            next(err)
33          }
```

Soluzione Applicata ed Implementazione della Remediation:

Per poter ovviare a questo problema la soluzione opportuna, quindi, resta un controllo di validazione che interpreta solo JSON e restituisce errore per qualunque tentativo di inserimento non opportuno.
Info: La seguente vulnerabilità e stata segnalata con le seguenti CVE: **CVE-2017-9807, CVE-2017-9802**

```
export function b2bOrder () {
  return ({ body }: Request, res: Response, next: NextFunction) => {
    const orderLinesData = body.orderLinesData  // stringa JSON inviata dal client

    if (orderLinesData) {
      try {
        JSON.parse(orderLinesData) // SICURO: interpreta solo JSON
        // Se non è JSON valido → errore 500
        next(err)
        return
```

4.5 Caso Tecnico 5: Denial of Service (DoS)

- **File Coinvolto:** *juice-shop-master/server.ts*
- **Identificazione CWE: CWE-400: Uncontrolled Resource Consumption, CWE-770 (Allocation of Resources Without Limits)**
- **Mappatura OWASP: A05:2021-Security Misconfiguration.**
- **Review priority:** Medium
- **Tipo: Security Hotspots**

Descrizione della Vulnerabilità:

In questo caso tecnico SonarQube segnala uno security hotspot sulla configurazione di multer, a riga 694;
Il problema è che non è definito un limite alla dimensione dei file caricabili, esponendo l'applicazione a rischio Denial of Service (DoS) tramite upload di file eccessivamente grandi.

```
694  const uploadToDisk = multer({
695    storage: multer.diskStorage({
696      destination: (req: Request, file: any, cb: any) => {
697        const isValid = mimeTypeMap[file.mimetype]
698        let error: Error | null = new Error('Invalid mime type')
699        if (isValid) {
700          error = null
701        }
702        cb(error, path.resolve('frontend/dist/frontend/assets/public/images/uploads/'))
703      },
704      filename: (req: Request, file: any, cb: any) => {
705        const name = security.sanitizeFilename(file.originalname)
706          .toLowerCase()
707          .split(' ')
708          .join('-')
709        const ext = mimeTypeMap[file.mimetype]
710        cb(null, name + '-' + Date.now() + '.' + ext)
711      }
712    })
713  })
```

Soluzione Applicata ed Implementazione della Remediation:

Per risolvere questo hotspots occorre assolutamente implementare un limite massimo tramite la funzione fileSize, in questo caso a riga 712 assegnando come limite 8MB che è lo standard consigliato dall'OWASP.

```
702        cb(error, path.resolve('frontend/dist/frontend/assets/public/images/uploads/'))
703      },
704      filename: (req: Request, file: any, cb: any) => {
705        const name = security.sanitizeFilename(file.originalname)
706          .toLowerCase()
707          .split(' ')
708          .join('-')
709        const ext = mimeTypeMap[file.mimetype]
710        cb(null, name + '-' + Date.now() + '.' + ext)
711      },
712      limits: { fileSize: 8000000 } //8MB
713    })
714  })
```

5. CONSIDERAZIONI FINALI

L'analisi SAST condotta sul repository OWASP Juice Shop tramite SonarQube ha permesso di identificare 14 Open Issue e 107 Security Hotspots. In questo report sono state isolate 5 tra le problematiche reali più gravi (2 aperte e 3 hotspots), scartando tra questi i falsi positivi.
I risultati emersi confermano un pattern ricorrente e ben noto nel settore: **la separazione tra logica applicativa e dati sensibili** viene spesso trascurata in fase di sviluppo, accumulando debito di sicurezza.

Le criticità rilevate coprono un range di attacco molto ampio: dall'esposizione di segreti crittografici nel backend, alle credenziali in chiaro nel frontend compilato, fino ad arrivare a vulnerabilità attive come XSS e RCE che mettono a rischio non solo l'infrastruttura, ma anche gli utenti finali dell'applicazione**. È fondamentale sottolineare che nessuna di queste falle richiede competenze avanzate per essere sfruttata:** un developer tools del browser o una semplice lettura del repository sarebbero sufficienti.
Questo report di assessment ha quindi dimostrato il valore strategico dell'analisi SAST e come si può utilizzare SonarQube per facilitare e mettere in sicurezza il ciclo di sviluppo di un Software.

Criticità Rilevate	OWASP	TIPO	SEVERITY	CHECK MANUALE
Private Key	A02:2021 – Cryptographic Failures	Security Open Issue	**CRITICAL**	**True Positive**
User Password	A07:2021 – Identification and Authentication Failures	Security Open Issue	**CRITICAL**	**True Positive**
Vulnerability XSS	A03:2021 – Injection	Security Hotspot	**HIGH**	**True Positive**
Remote Code Execution (RCE)	A03:2021 – Injection	Security Hotspot	**MEDIUM**	**True Positive**
Denial of Service (DoS)	A05:2021-Security Misconfiguration.	Security Hotspot	**MEDIUM**	**True Positive**

Analisi SAST sul Repository OWASP Juice Shop.
Studente: **Stefano Panosetti**
2026